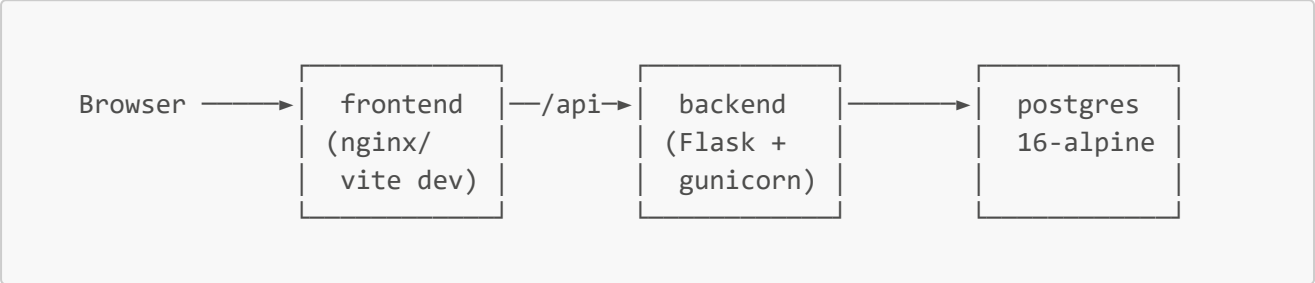
Exercise 1.1 — Multi-Service Docker Environment

A three-service stack (Flask API, React/Vite frontend, PostgreSQL) with separate dev/staging/production configurations, health checks, logging, and basic monitoring hooks.

> Screenshot placeholders are marked 📸 `[SCREENSHOT: ...]` throughout — replace each with `![alt](./screenshots/filename.png)` once captured, and drop the PNGs in `docs/screenshots/`.

Architecture



- **backend/**: Flask API. `python app.py` in dev, `gunicorn` (4 workers) in production. `/api/health` checks the DB connection for readiness probes.
- **frontend/**: Vite + React + TypeScript. `vite dev` server locally; built to static assets and served by nginx in production, which also reverse-proxies `/api/*` to the backend so the browser only talks to one origin.
- **postgres/**: Official `postgres:16-alpine` image, seeded by `postgres/init.sql` on first boot.

Environments

Compose uses a base file plus an override per environment:

Environment	Command
Development	`docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build`
Staging	`docker compose -f docker-compose.yml -f docker-compose.staging.yml up --build -d`
Production	`docker compose -f docker-compose.yml -f docker-compose.prod.yml up --build -d`

Differences by environment:

- **Development**: builds the `development` Dockerfile stage, bind-mounts source for live reload, publishes backend (`5000`), frontend (`5173`),

and postgres (`5432`) ports directly, verbose logging.
- **Staging**: builds the `production` stage (so you're testing the real artifact), single replica per service, resource limits applied, `restart: unless-stopped`, frontend published on `8080`.
- **Production**: `production` stage, 3 backend replicas / 2 frontend replicas, `restart: always` with a failure-based restart policy, DB password supplied via a Docker secret instead of a plain env var, backend port is **not** published (only reachable via the frontend's nginx proxy), frontend published on `443`.

Before running production, copy the example secret and replace it:

```
cp secrets/pg_password.txt.example secrets/pg_password.txt
# edit secrets/pg_password.txt with a real password, then update
# POSTGRES_PASSWORD_FILE usage / DATABASE_URL to match your secrets manager
```

Health checks

Every service defines a `HEALTHCHECK` (Docker-level) plus, for Postgres, a `pg_isready` compose healthcheck. `backend` and `frontend` don't start their dependent services until `postgres` reports healthy (`depends_on: condition: service_healthy`).

Check status:

```
docker compose ps
```

Logging

All services use the `json-file` driver capped at `10m` per file, 3 files retained (`x-logging` anchor in `docker-compose.yml`), to prevent unbounded log growth on the host. Backend logs structured lines to stdout with timestamp/level/logger name; nginx access/error logs also go to stdout/stderr so `docker compose logs -f <service>` captures everything — this is what you'd point Prometheus/Loki/ELK at in Module 1.2.

Monitoring hooks

- `/api/health` on the backend returns `200` with `{"service": "ok", "db": "ok"}` when healthy, `503` when the DB is unreachable — wire this into an external uptime check or a Kubernetes readiness probe later.
- `/healthz` on the frontend nginx container is a lightweight liveness endpoint separate from the app itself.

- Container `HEALTHCHECK` status is visible via `docker inspect` or `docker compose ps` and is what orchestrators (Swarm/K8s) use for restart decisions.

Local quickstart

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
# frontend: http://localhost:5173
# backend:  http://localhost:5000/api/health
```

Steps & screenshots — proof of work

Run against the **dev** stack (`docker-compose.yml` + `docker-compose.dev.yml`), on Docker Desktop / WSL2, Windows. Two real issues were hit and fixed along the way — documented below rather than smoothed over, since that's more useful evidence than a silent happy path.

1. Confirm the environment

```
docker info
```

Docker Engine 29.2.1, WSL2 backend, 8 CPUs, 9.7 GiB memory — server block confirmed healthy before starting anything.
📸 `[SCREENSHOT: docker info — Server block]`

2. Build and start the stack

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
```

All three images built clean (postgres pulled, backend and frontend built from source, ~42s each). Postgres reported `database system is ready to accept connections` and hit `Healthy` before backend/frontend started, confirming `depends_on: condition: service_healthy` worked as designed.
📸 `[SCREENSHOT: terminal — full build/startup log]`

3. Check container health — first pass

```
docker compose ps
```

Result: `backend` and `postgres` **healthy**, `frontend` **unhealthy**.
📸 `[SCREENSHOT: docker compose ps — frontend unhealthy]`

4. Root-cause the frontend health check

```
docker inspect --format='{{json .State.Health}}' exercise-11-frontend-1
```

Output showed a consistent `wget: can't connect to remote host: Connection refused` on every attempt (not a timeout — a refusal), pointing at DNS/ resolution rather than the app being slow to start. Confirmed with two manual requests from inside the container:

```
docker exec exercise-11-frontend-1 wget -qO- http://127.0.0.1:5173 -T 3   # succeeded, returned Vite HTML
docker exec exercise-11-frontend-1 wget -qO- http://localhost:5173 -T 3   # failed, same "Connection refused"
```

Root cause: Alpine's musl libc resolves `localhost` to the IPv6 loopback (`::1`) first; Vite's dev server wasn't reachable there, only on the IPv4 loopback. The app was fine — only the health check's target address was wrong.

🖼 [SCREENSHOT: docker inspect Health JSON showing the refused log]

🖼 [SCREENSHOT: the two wget test results, side by side]

Fix — `frontend/Dockerfile`, `HEALTHCHECK` line:

```
- CMD wget -qO- http://localhost:5173 || exit 1
+ CMD wget -qO- http://127.0.0.1:5173 || exit 1
```

5. Rebuild frontend and recheck health

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build -d frontend
docker compose ps
```

Result: all three services **healthy**.

🖼 [SCREENSHOT: docker compose ps — all healthy, post-fix]

6. Verify the frontend in a browser — first pass

Opened `http://localhost:5173`. Page rendered, but showed **"API says: API unreachable"** instead of the expected message — container health didn't mean the feature worked end to end.

🖼 [SCREENSHOT: browser — "API unreachable"]

7. Root-cause the API call

```
curl -i http://localhost:5173/api/hello
```

Returned `200 OK`, but with `Content-Type: text/html` and Vite's `index.html` shell in the body — not JSON from the backend. **Root cause:** the dev Vite server had no proxy configured for `/api/*` (unlike the production nginx config, which does proxy it), so the browser's fetch silently hit Vite's SPA fallback instead of erroring loudly.
📸 `[SCREENSHOT: curl output showing HTML instead of JSON]`

Fix — `frontend/vite.config.ts`, added a dev-server proxy:

```
  server: {
    host: true,
    port: 5173,
+   proxy: {
+     "/api": {
+       target: "http://backend:5000",
+       changeOrigin: true,
+     },
+   },
  },
```

`backend:5000` resolves via Docker Compose's internal DNS on the `appnet` network — the same pattern the production nginx config uses.

8. Rebuild frontend and recheck the browser

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build -d
frontend
```

Reloaded `http://localhost:5173` — page showed **"API says: Hello from the Flask API"**, confirming the full browser → Vite proxy → backend → response path.
📸 `[SCREENSHOT: browser — "Hello from the Flask API"]`

9. Verify backend health endpoint directly

```
curl -i http://localhost:5000/api/health
```

`200 OK`, `{"service": "ok", "db": "ok"}` — the `db: ok` field confirms the Flask container has a live connection to Postgres, not just that

Flask itself is running.

📸 [SCREENSHOT: curl output — service/db both "ok"]

10. Capture logs as monitoring evidence

```
docker compose logs backend --tail 20
docker compose logs postgres --tail 10
```

Backend log includes the two `/api/hello` requests from the browser test (`172.18.0.4` — the frontend container's internal IP, arriving via the proxy). Postgres log shows a real autovacuum checkpoint cycle, confirming it was alive and doing normal housekeeping, not just idling.

📸 [SCREENSHOT: backend + postgres log tails]

11. Tear down

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml down
```

Clean shutdown; the `pgdata` named volume persists across runs, so the next `up` won't re-run `init.sql`.

📸 [SCREENSHOT: terminal — clean shutdown]

Issues found & fixed — summary

#	Symptom	Root cause	Fix
1	`frontend` container stuck `unhealthy` despite app working	Alpine/musl resolves `localhost` to IPv6 first; `wget` health check couldn't reach Vite there	`HEALTHCHECK` target changed `localhost` → `127.0.0.1` in `frontend/Dockerfile`
2	Frontend page showed "API unreachable" even with all containers healthy	No dev-server proxy for `/api/*`; requests fell through to Vite's SPA `index.html` fallback (`200`, HTML, not JSON)	Added `server.proxy["/api"] → http://backend:5000` in `frontend/vite.config.ts`

Both bugs were "container healthy, feature still broken" cases — a reminder that infra-level health checks and actual functional correctness are two different questions, and this run checked both.